

This is a fantastic project architecture. Combining declarative automation with Policy as Code (PaC) is exactly the kind of "fail-proof" methodology that stands out on a GitHub portfolio, especially when demonstrating enterprise-grade reliability and security-by-default practices.

Here is a breakdown of how to structure this pipeline, the best tools for the job, and how to elevate it further.

The Architecture: "Secure-by-Default" Deployment

To make GitHub the single source of truth and enforce policies before a single VM is spun up, you will want a Continuous Integration/Continuous Deployment (CI/CD) pipeline.

Here is the step-by-step workflow:

1. **Version Control (GitHub):** You write your Terraform configuration (main.tf) and your PaC rules, then commit them to a GitHub repository.
2. **CI/CD Trigger (GitHub Actions):** Opening a Pull Request triggers a GitHub Actions workflow.
3. **The PaC Gatekeeper:** The workflow runs your PaC tool against the Terraform code. It checks for security implementations (e.g., "Are SSH keys securely configured?" or "Are resource limits defined?").
4. **Approval & Merge:** If the code violates policy, the pipeline fails, and the code cannot be merged. If it passes, the PR can be merged.
5. **Deployment:** Once merged, Terraform applies the configuration to Multipass.

Recommended Tooling

- **Terraform & Multipass:** Because Multipass runs locally on your machine, HashiCorp doesn't have an official tier-one provider for it. However, you can use a community provider like larstobi/multipass to manage the instances declaratively.
- **Policy as Code: * Open Policy Agent (OPA) / Conftest:** You write custom policies in a language called Rego. For example, you can write a policy stating: *"Deny any Multipass VM deployment that requests more than 4GB of RAM or doesn't include a cloud-init file for security hardening."*

- **Checkov:** An alternative that comes with hundreds of built-in security policies for Terraform, though it is more tailored to major cloud providers (AWS/GCP/Azure) than local Multipass environments (which we are using).
- **The Bridge (Self-Hosted Runner):** Since GitHub Actions runs in the cloud, it cannot natively "see" the Multipass daemon running on your local machine. You will need to install a **GitHub Self-Hosted Runner** on your primary machine (like your Ubuntu command center). This allows GitHub to securely execute the Terraform commands directly in your local environment.

Here is the blueprint for Phase 1.

For this project we are using MacOs as the command centre. If you don't have Homebrew installed yet, you can grab it from brew.sh. Otherwise, open your terminal and run these commands to get your macOS environment ready.

Step 1: Installing the Toolchain (macOS)

1. Install Git & Python macOS usually comes with these, but it is best practice to manage them via Homebrew to keep them updated:

Bash

```
brew install git python
```

2. Install Multipass We will install Multipass as a macOS cask application:

Bash

```
brew install --cask multipass
```

3. Install Terraform HashiCorp maintains an official Homebrew tap:

Bash

```
brew tap hashicorp/tap
```

```
brew install hashicorp/tap/terraform
```

4. Install Checkov You can install Checkov directly through Homebrew as well:

Bash

```
brew install checkov
```

You can verify the installations just like before by running `terraform -version`, `multipass version`, and `checkov --version`.

Step 2: Structuring the Repository

A clean, modular repository structure is critical for maintaining high-availability architectures as they scale. We will create a dedicated workspace that separates your infrastructure logic, your policies, and your CI/CD pipelines.

Run this block in your terminal to generate the entire folder structure instantly:

Bash

```
# Create the root project directory and move into it
```

```
mkdir -p failproof-infrastructure && cd failproof-infrastructure
```

```
# Initialize Git
```

```
git init
```

```
# Create the folder hierarchy
```

```
mkdir -p .github/workflows
```

```
mkdir -p terraform
```

```
mkdir -p policies
```

Create the foundational files

```
touch .github/workflows/pac-pipeline.yml
```

```
touch terraform/main.tf
```

```
touch terraform/variables.tf
```

```
touch terraform/providers.tf
```

```
touch policies/custom_rules.yaml
```

```
touch .gitignore
```

Your project tree now looks like this:

Plaintext

failproof-infrastructure/

```
├── .github/
│   └── workflows/
│       └── pac-pipeline.yml    # The GitHub Actions CI/CD logic
├── policies/
│   └── custom_rules.yaml      # Where we will write specific Checkov
rules
├── terraform/
│   ├── main.tf               # The actual Multipass VM deployment logic
│   ├── providers.tf          # Terraform provider configurations
│   └── variables.tf           # Dynamic variables for scalability
└── .gitignore                 # To keep state files out of Git
```

Step 3: The First Workflow (The Provider)

Before we build the VMs or the policies, Terraform needs to know *how* to talk to Multipass.

Open `terraform/providers.tf` in your preferred text editor and paste the following code. This pulls in the community-supported Multipass provider:

Terraform

```
terraform {  
  required_providers {  
    multipass = {  
      source = "larstobi/multipass"  
      version = "~> 1.4.2"  
    }  
  }  
}
```

```
provider "multipass" {}
```

Next, let's secure the `.gitignore`. Add these lines so you don't accidentally commit sensitive local state files to GitHub:

Plaintext

```
# Local .terraform directories
```

```
**/.terraform/*
```

```
# .tfstate files
```

```
*.tfstate
```

```
*.tfstate.*
```

```
# Crash log files
```

```
crash.log
```

```
# Ignore any .tfvars files that are generated automatically
```

```
*.tfvars
```

Terraform VM configuration (main.tf) Let's build out the main.tf and variables.tf files. To ensure this project serves as a strong demonstration of enterprise-grade reliability and scalability, we are going to design it with High Availability (HA) in mind right from the start, rather than just spinning up a single, isolated VM.

1. Defining the Variables (variables.tf)

Open your terraform/variables.tf file and add this:

Terraform

```
variable "node_count" {
```

```
    description = "The number of instances to deploy for high availability"
```

```
    type        = number
```

```
    default     = 2
```

```
}
```

```
variable "memory_limit" {
```

```
description = "RAM allocated per node"

type      = string

default   = "2G"

}
```

2. Defining the Infrastructure (main.tf)

Now, open terraform/main.tf. This code will use the variables we just created to spin up multiple VMs simultaneously.

Terraform

```
resource "multipass_instance" "portfolio_node" {

  # The count meta-argument loops through and creates multiple VMs

  count = var.node_count

  # Dynamically names them: portfolio-node-1, portfolio-node-2

  name = "portfolio-node-${count.index + 1}"

  cpus = 2

  memory = var.memory_limit

  disk = "10G"

  # NOTE: We will uncomment this in the next phase when we introduce
  # security policies that mandate a cloud-init hardening script.
  # cloudinit_file = "${path.module}/cloud-init.yaml"

}
```

```
output "node_ip_addresses" {  
    description = "A map of node names to their IP addresses"  
  
    value = { for node in multipass_instance.portfolio_node : node.name =>  
node.ipv4 }  
  
}
```

1. The Security Bootstrap (cloud-init.yaml)

Terraform is great at creating the VM (the "hardware"), but cloud-init is the industry standard for securely configuring the OS (the "software") the moment it boots up.

In your root failproof-infrastructure directory, open terraform/cloud-init.yaml and paste this configuration:

YAML

```
# cloud-init.yaml  
  
# Security Hardening & Bootstrapping  
  
package_update: true  
  
package_upgrade: true  
  
  
users:  
  
  - default  
  
  - name: deployment-admin  
  
    gecos: Deployment Administrator  
  
    sudo: ALL=(ALL) NOPASSWD:ALL
```


groups: users, admin

shell: /bin/bash

lock_passwd: true

ssh_authorized_keys:

Replace this with your Mac's actual public SSH key (found in ~/.ssh/
id_rsa.pub)

- ssh-rsa YOUR_PUBLIC_SSH_KEY_HERE

Disable basic password authentication to force SSH key usage

ssh_pwauth: false

runcmd:

- echo "Hardening complete. Node ready for clustering." >> /var/log/node-
bootstrap.log

*Note: Make sure to go back to your terraform/main.tf file and uncomment the
cloudinit_file line so Terraform knows to use this file!*

2. The Policy Gatekeeper (custom_rules.yaml)

Now we need to write the Policy as Code (PaC) rule. We want Checkov to scan our Terraform code and definitively state: *"If a developer tries to deploy a Multipass VM without a cloud-init security file attached, fail the deployment."*

Open policies/custom_rules.yaml and paste this Checkov policy:

YAML

metadata:

name: "Ensure Multipass instances use cloud-init for security hardening"

id: "CKV_CUSTOM_1"

category: "GENERAL_SECURITY"

definition:

cond_type: "attribute"

resource_types:

- "multipass_instance"

attribute: "cloudinit_file"

operator: "exists"

What this does: * **resource_types:** Tells Checkov to only look at multipass_instance blocks in your Terraform code.

- **attribute & operator:** Checks if the cloudinit_file attribute actually exists. If someone comments it out or deletes it, Checkov will throw a CKV_CUSTOM_1 error and block the deployment.

See it in Action

You can actually test this locally on your Mac right now without even spinning up the VMs. Checkov scans the *code*, not the live environment.

Run this command in your terminal from the root of your project:

Bash

```
checkov -d terraform/ --external-checks-dir policies/
```

If your main.tf has the cloudinit_file line uncommented, Checkov will pass it with flying colors. If you comment that line out and run the scan again, you will see Checkov instantly block it.

This is a very common roadblock! If you can't find ~/.ssh/id_rsa.pub, it usually means one of two things: either you have never generated an SSH key on

this Mac, or you generated a more modern key that goes by a different name (like id_ed25519).

Here is exactly how to find it or create a new one.

Step 1: Check what you already have

Let's look inside your hidden SSH folder. Run this command in your Mac terminal:

Bash

```
ls -al ~/.ssh
```

- **If you see files ending in .pub** (like id_ed25519.pub or id_rsa.pub), you already have a key! You can skip to Step 3.
- **If you get a "No such file or directory" error**, or if the folder is empty/has no .pub files, you need to generate a new key. Move to Step 2.

Step 2: Generate a new SSH Key (If needed)

The tech industry has largely moved away from rsa and now uses ed25519 because it is faster and far more secure. Let's generate a modern key.

Run this command (you don't need to change anything, just press Enter through all the prompts to accept the defaults):

Bash

```
ssh-keygen -t ed25519 -f ~/.ssh/id_ed25519 -N ""
```

(Note: The -N "" part creates the key without a passphrase, which is ideal for automated CI/CD and local testing so it doesn't prompt you for a password every time).

Step 3: Copy the Public Key

Now we need to grab the exact text of that public key to paste into your cloud-init.yaml file.

Macs have a brilliant built-in command called pbcopy that copies file contents directly to your clipboard. Run the command that matches the file you found or created:

If you generated the modern key (or found it in Step 1):

Bash

```
pbcopy < ~/.ssh/id_ed25519.pub
```

If you found an older RSA key in Step 1:

Bash

```
pbcopy < ~/.ssh/id_rsa.pub
```

Step 4: Update your cloud-init.yaml

Your key is now copied to your Mac's clipboard!

Open your terraform/cloud-init.yaml file again. Delete YOUR_PUBLIC_SSH_KEY_HERE and simply hit Cmd + V to paste your actual key.

The line should look something like this once you paste it:

YAML

```
- ssh-ed25519 AAAAC3NzaC1lZDI1NTE5AAAAI... (followed by a bunch of random letters)
```

Once that is pasted and saved, your security bootstrap is officially complete.

To test this policy scan locally to see the output, and move directly into building the **GitHub Actions pipeline (pac-pipeline.yml)** so this scan happens automatically in the cloud every time you commit code.

Part 1: What to Expect Locally

If you run `checkov -d terraform/ --external-checks-dir policies/` in your terminal right now, you will see a summary output that looks something like this:

Plaintext

```

      _      _
    _||_  _  _||_
 / _|'_\/_V_|//_\|//
|(_||||_/_/(_| <(_)\V/
 \_|_|_|_|_|_|_|_|_|/

```

terraform scan results:

Passed checks: 1, Failed checks: 0, Skipped checks: 0

Check: CKV_CUSTOM_1: "Ensure Multipass instances use cloud-init for security hardening"

PASSED for resource: multipass_instance.portfolio_node

File: /main.tf:1-12

If it says **PASSED**, your code is secure and ready for the cloud.

Part 2: Building the GitHub Actions Pipeline

We are going to create a CI/CD pipeline that runs in GitHub's cloud. Because Checkov only reads *text* (your `.tf` files) and doesn't actually spin up the VMs, this scan can run entirely on GitHub's free cloud servers without needing to connect to your Mac.

Open the file we created earlier at `.github/workflows/pac-pipeline.yml` and paste in this configuration:

YAML

name: Infrastructure Policy as Code Gatekeeper

1. The Triggers: When does this run?

on:

push:

branches: ["main"]

pull_request:

branches: ["main"]

2. The Jobs: What does it do?

jobs:

security-scan:

name: Checkov Security Scan

runs-on: ubuntu-latest # Runs on a fresh GitHub-hosted cloud server

steps:

Step A: Download your code into the runner

- name: Checkout Repository

uses: actions/checkout@v4

Step B: Run the official Checkov GitHub Action

- name: Run Checkov against Infrastructure

uses: bridgecrewio/checkov-action@master

with:

directory: terraform/

external_checks_dir: policies/

framework: terraform

This ensures the pipeline FAILS if a policy is violated

soft_fail: false

How this Gatekeeper works:

1. **The Trigger:** Whenever you try to push code or merge a Pull Request to the main branch, GitHub intercepts it.
2. **The Cloud Server:** GitHub spins up a temporary ubuntu-latest server in the cloud.
3. **The Scan:** It downloads your repository and runs the exact same Checkov command you ran locally.
4. **The Gate:** Because we set `soft_fail: false`, if someone deletes the `cloudinit_file` line in `main.tf`, Checkov throws a red "X" and GitHub physically blocks the code from being merged. Your main branch is protected.

You now have a fully functional Policy as Code gatekeeper!

To see this pipeline come to life, we need to push this repository to GitHub. You should have an empty GitHub repository created for this project, and need to quickly link your local folder to a new remote repo.

You have officially built the "Gatekeeper" phase of this project! The cloud CI/CD pipeline is protecting your codebase. Now, it is time to shift from the cloud back to your MacBook and actually bring this infrastructure to life.

We are done with the *foundation*. But right now, this deployment will just spin up two highly secure, but completely empty, Linux boxes. To make this a true

"fail-proof", we still need to deploy a workload onto them and make them resilient.

Here is exactly how we test the deployment locally right now.

Phase 1: The Local Deployment Test

Since your main.tf code passed the security scan in GitHub, we know it is safe to execute. Open your Mac terminal, ensure you are in the failproof-infrastructure directory, and run the "Holy Trinity" of Terraform commands:

1. Initialize the Workspace

Bash

```
terraform init
```

What this does: This reads your providers.tf file and downloads the larstobi/multipass plugin we discussed earlier so Terraform can talk to your Mac.

2. Preview the Execution Plan

Bash

```
terraform plan
```

What this does: Terraform compares your code to reality. It will print out a list showing exactly what it intends to build (two Multipass VMs, allocating 2GB of RAM and 2 CPUs to each) without actually building them yet.

3. Build the Infrastructure

Bash

```
terraform apply
```

What this does: It will ask you to type yes to confirm. Once you do, Terraform will reach out to Multipass and physically spin up the virtual machines, applying your secure cloud-init.yaml profile to both of them. *(Note: This will temporarily use 4GB of your Mac's RAM).*

4. Verify the Build

To prove Terraform did its job, ask Multipass to show you the running systems:

Bash

```
multipass list
```

You should see portfolio-node-1 and portfolio-node-2 running securely!

Since we need to update our cloud-init.yaml to install Kubernetes (K3s) and make this a Highly Available (HA) cluster, let's use that as our real-world feature to practice this exact workflow.

Here is how we do it, step-by-step.

Step 1: Create the Working Branch

Open your Mac terminal (ensure you are in the failproof-infrastructure root folder) and run this command:

Bash

```
git checkout -b feature/failproof-infra
```

- **What this does:** It takes a perfect snapshot of your main branch right now and switches you into a safe, isolated parallel universe called feature/k3s-clustering. Any changes you make here will not affect main.

Step 2: Make the Code Change

Now we do the actual engineering work. Open your terraform/cloud-init.yaml file. We are going to add a command to install K3s so the VMs automatically configure themselves as a cluster upon booting.

Update the bottom of the file so the runcmd block looks like this:

YAML

runcmd:

```
# Install K3s (Lightweight Kubernetes)

- curl -sfL https://get.k3s.io | sh -

- echo "Hardening complete. K3s installed and node ready for clustering."
>> /var/log/node-bootstrap.log
```

Save the file.

Similarly for Nginx:

runcmd:

```
# Install Nginx web server

- apt-get install -y nginx

- echo "Hardening complete and Web Server running." >> /var/log/node-
bootstrap.log
```

Step 3: Push the Branch to GitHub

Your change is saved locally on your Mac. Now, we need to send this new branch up to GitHub so we can propose it to the team (and the automated gatekeeper).

Run these three commands:

Bash

```
git add terraform/cloud-init.yaml
```

```
git commit -m "feat: bootstrap K3s installation for HA clustering and nginx web server on all nodes"
```

```
git push -u origin feature/failproof-infra
```

Step 4: Open the Pull Request

Now we step out of the terminal and into the cloud.

1. Go to your repository on **GitHub.com**.
2. You will see a green banner at the top that says feature/k3s-clustering had recent pushes. Click the **Compare & pull request** button next to it.
3. You are now officially proposing your change! Leave a brief description if you want, and click the green **Create pull request** button.

Step 5: Watch the Gatekeeper Work

The moment you click that button, look slightly down the Pull Request page. You will see a yellow dot spinning.

This is the Checkov GitHub Action running against your new feature/k3s-clustering branch. It is verifying that your new Kubernetes installation code didn't accidentally delete the SSH keys or violate any security rules.

Wait about 15 seconds. Once Checkov finishes scanning, it will turn into a green checkmark indicating **"All checks have passed."**

Because the automated Health Inspector gave you the green light, it is now officially safe to click the **Merge pull request** button. Doing so will permanently add your K3s upgrade into the official main recipe!

Right now, your Terraform code is configured to build two highly secure VMs and automatically install K3s (Kubernetes) and Nginx on them. But simply having servers running isn't enough. We need to see them act as a resilient system.

We are going to deploy an application, intentionally break it, and watch the system fix itself automatically without human intervention.

Here are your next steps to get the environment ready for the test:

Step 1: Rebuild the Real Infrastructure

Since we wiped your local folder clean during the onboarding simulation, you need to tell Terraform to reach out to Multipass and physically build the servers using your newly approved, master recipe.

Open your terminal and run:

Bash

```
cd terraform
```

```
terraform init
```

```
terraform apply
```

(Type yes to confirm. This might take a minute or two as it downloads the VMs and installs Kubernetes and Nginx).

Step 2: Enter the Command Center

Once the VMs are running, we need to log into the first node so we can start giving commands to the Kubernetes cluster. Multipass makes this incredibly easy.

Run this command to securely SSH into your first server:

Bash

```
multipass shell portfolio-node-1
```

(You will see your terminal prompt change from MacBook to ubuntu@portfolio-node-1. You are now inside the cloud!)

Step 3: Verify the Cluster Health

As an SRE, you never assume a deployment worked just because there were no errors. You always verify. Let's ask Kubernetes if it is awake and healthy.

Run this command inside the node:

Bash

```
sudo k3s kubectl get nodes
```

You should see portfolio-node-1 listed with a status of **Ready**.

If you see that "Ready" status, we are primed for the final experiment. Our next step is to deploy a "Self-Healing Workload" into this cluster—a web app that automatically restarts itself if it crashes.

This is a massive milestone. You just used code to automatically provision hardware, configure security, and bootstrap a production-grade Kubernetes control plane.

Now, we get to the fun part. This is where we test the core of self-healing container logic.

In a traditional IT environment, if a web server crashes, an alarm goes off, an engineer wakes up at 3:00 AM, logs in, and manually restarts the service.

In an environment, the system is **declarative**. You tell Kubernetes, *"I always want exactly one web server running."* If the server crashes, Kubernetes notices the reality no longer matches your declaration, and it fixes it automatically within milliseconds.

Let's deploy a web app and intentionally destroy it to watch this happen.

Step 1: Declare the Workload

You are currently logged into portfolio-node-1. We are going to tell Kubernetes to spin up a deployment called failproof-web using a standard Nginx container image.

Run this command:

Bash

```
sudo k3s kubectl create deployment failproof-web --image=nginx
```

(Kubernetes says: "Deployment created.")

Step 2: Verify the Reality

Let's look at the actual container (called a "Pod" in Kubernetes) that just spun up.

Run this command:

Bash

```
sudo k3s kubectl get pods
```

You should see a list with one item in it. It will look something like this:

```
failproof-web-5c8b556c8f-xyz12 1/1 Running 0 15s
```

Copy the exact name of your pod (that long string of letters and numbers under the "NAME" column). You will need it for the next step.

Step 3: The Chaos Experiment

It is time to be the villain. We are going to act like a critical hardware failure or a massive bug just completely destroyed your application. We are going to delete the pod.

Run this command, replacing YOUR-POD-NAME with the exact name you copied:

Bash

```
sudo k3s kubectl delete pod YOUR-POD-NAME
```

Step 4: Watch it Heal

The second you hit enter, Kubernetes deletes the container.

Immediately run your verification command again:

Bash

```
sudo k3s kubectl get pods
```

What You Just Witnessed

If you ran that last command fast enough, you likely saw the old pod say Terminating, and right beneath it, a brand new pod with a completely different random name saying ContainerCreating or Running.

You didn't have to tell Kubernetes to build a new one. It constantly runs a control loop, saw that the number of active web servers dropped from 1 to 0, panicked, and instantly rebuilt it to maintain the state you declared in Step 1.

The Firewall Problem (Why GitHub can't touch your Mac)

Right now, your GitHub Action (the Checkov scanner) runs on a temporary Microsoft server in the cloud.

If we tell that cloud server to run terraform apply, it will look around its own environment, realize it doesn't have Multipass installed, and fail.

Even worse, that cloud server cannot reach through your home Wi-Fi router, bypass your Mac's firewall, and command your local hypervisor to build VMs. Your Mac is physically isolated from GitHub.

How Enterprise SREs Build the "Auto-Deploy" (The Real World)

In a real company, the infrastructure isn't living on a laptop; it lives in AWS, Azure, or Google Cloud. Because those environments are accessible via the internet, GitHub *can* reach them securely.

If we were deploying to AWS instead of your Mac, here is exactly how we would upgrade your pipeline to achieve your vision:

1. The Credentials:

We would give GitHub Actions an encrypted "API Key" that acts as a VIP badge to the company's AWS account.

2. The Phase 2 Pipeline (The Deployer):

We would write a second GitHub Actions workflow file. Instead of running when a Pull Request is *opened*, we configure it to run only when code is *merged into main*.

It would look exactly like this in code:

YAML

on:

push:

branches:

- main # This trigger only fires AFTER the Senior SRE merges the code!

jobs:

deploy-to-production:

runs-on: ubuntu-latest

steps:

- name: Download Code

uses: actions/checkout@v3

- name: Login to AWS

uses: aws-actions/configure-aws-credentials@v2

with:

aws-access-key-id: \${ secrets.AWS_ACCESS_KEY }

... other security tokens

- name: Setup Terraform

uses: hashicorp/setup-terraform@v2

- name: The Auto-Deploy

run: |

terraform init

terraform apply -auto-approve

The Secret: "GitOps"

Notice the -auto-approve flag at the very end of that script.

Normally, Terraform pauses and makes you type yes to confirm you want to build the servers. But because a Senior SRE already reviewed the code, checked the Checkov scan, and clicked "Merge," the human approval is already finished. We tell Terraform to skip the prompt and just build it instantly.

This philosophy is called **GitOps**. The Git repository (GitHub) is the absolute single source of truth. The moment the repository changes, the physical servers instantly morph to match it.